Anchoring Marketplace State

Two economies on one emission schedule, replicated by hand. Neither domain's contracts can read the other, so every claim tying emission to compute demand rests on an off-chain component.

The split. Capital lives on Ethereum, where `DistributorV2` splits the period's MOR across deposit pools by Aave yield. Compute lives on Base, in one EIP-2535 diamond holding its own copy of the schedule: a `Pool[]` array written by `setPoolConfig` under `onlyOwner`, whose `@dev` line says the values "should be the same as in Ethereum L1 Distribution contract" (`SessionRouter.sol:57`, `SessionStorage.sol:16`). MOR crosses as a LayerZero OFT and a Python service sits between. Nothing checks the mirror. Nor is either Base payout a mint: providers get `safeTransferFrom(fundingAccount, provider, amount)` (`SessionRouter.sol:397`), builder stakers `safeTransferFrom(treasury, staker, toClaim)` (`BuilderSubnets.sol:335`), so payment depends on a balance and an approval held off-chain. "24% of emission" is a policy about a schedule; at the contract level it is a treasury someone tops up.

The coupling variable, and the mirror's error. `stakeToStipend` makes purchasable session time a function of `getComputeBalance`: cumulative compute emission minus `providersTotalClaimed`, the total drawn (`SessionRouter.sol:183, 392, 484`). That is the only place realised demand feeds back into what the market can sell, and Ethereum cannot read it. Both domains call the same deployed `LinearDistributionIntervalDecrease` with independently configured arguments, so relative error β in the decrease rate d gives relative error $\beta dn/(R_0 - dn)$ at day n, diverging as $R_0 - dn \to 0$. The mirror is least reliable exactly when the pool is smallest.

What must cross, and what must not. Minimising the crossed state is the design, so bound it: four scalars and two commitments per UTC day. Scalars are `providersTotalClaimed`, settled session-seconds, sessions closed, and min(balance, allowance) of the funding account, the real cap on payable capacity; commitments are a Merkle root over per-provider claim totals and a digest of the pool config in force. Excluded: sessions, bids, prices, prompts, model identity, telemetry. A daily digest of fixed size is the right object, not a message bus.

Verification, three levels, threat model named. The adversary is the diamond owner plus the funding-account operator: rewrite the pool config, repoint `fundingAccount`, stop funding, silently. *Attested digest with a challenge window* is nearly free and guarantees only that a false digest cannot pass unopposed. *Storage proof against a finalised Base output root* removes the trusted party with machinery Ethereum tooling ships, at the price of the rollup's fault window. *A native quorum certificate* removes that window and needs a compute domain that produces one. Lux cross-chain finality applies between Lux chains, not into a Base diamond; if that is the goal, LP-314 owns it.

What Morpheus gets

- A mirror that is checked, and drawn compute emission as a number governance reads, not infers.
- Owed and payable as separate visible quantities, so a shortfall surfaces before a provider does.

What we get, stated plainly

- A production anchoring target we did not design, a harder test than our own chains give.
- Enso routing measured against a demand series with provenance rather than a dashboard.

The cheap intermediate, and who takes the first step. Nothing above needs Lux. Three changes to Base and Ethereum as they stand: a `view` returning `keccak256` of the pool config each side holds, so anyone can diff them; `SessionClosed` (`ISessionRouter.sol:10`) widened to carry duration and amount, so demand is reconstructible from logs alone; balance and allowance exposed beside `getTodaysBudget`. The first is the whole first step and it belongs to a Morpheus contributor: add the digest, read `Distribution.pools(3)`, compare. An afternoon, no counterparty, no upgrade beyond a facet, and either the mirror holds or the divergence is a number. We publish the digest schema and its verifier.

Governance, which is why this matters. MRC authorises the 24% compute allocation, and the figure telling it whether that tracks real demand is `providersTotalClaimed`, in storage MRC cannot see. The last MRC commit is 2025-11-17. Anchoring does not fix the process; it gives it a number to check.

What this does not solve. Anchoring transports the compute domain's accounting faithfully; it does not make it true. A session an 8B model answered while a 70B was advertised commits like any other, so LP-313 is the prerequisite. It creates no demand, and a compromised owner can still rewrite state before a digest is taken.